

EXP 2: SVM and Random Forest for Classification

Aim

To build classification models using **Support Vector Machine (SVM)** and **Random Forest**, apply them to the Pet Adoption dataset, and evaluate their performance using **accuracy and confusion matrix**.

Algorithm

Part A — SVM

1. Import the required libraries.
2. Load and explore the Pet Adoption dataset.
3. Handle missing values.
4. Encode categorical variables.
5. Scale the features using StandardScaler.
6. Split the dataset into training and testing sets.
7. Initialize the SVM classifier using SVC().
8. Train the SVM model.
9. Predict the test data.
10. Calculate accuracy and confusion matrix.

Part B — Random Forest

1. Initialize the Random Forest classifier using RandomForestClassifier().
2. Train the model using the training data.
3. Predict the test data.
4. Calculate accuracy and confusion matrix.
5. Display the confusion matrices for SVM and Random Forest.

CODE:

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.svm import SVC

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix

print("Libraries imported successfully!")

df = pd.read_csv("pet_adoption.csv")

print("Dataset loaded successfully!")

print("Dataset shape:", df.shape)

display(df.head())

df["Breed"] = df["Breed"].fillna(df["Breed"].mode()[0])

df["Vaccinated"] = df["Vaccinated"].fillna(df["Vaccinated"].mode()[0])

df["Fee"] = df["Fee"].fillna(df["Fee"].median())

print("Missing values handled successfully!")

X = df.drop("AdoptionSpeed", axis=1)

y = df["AdoptionSpeed"]

print("Features shape:", X.shape)

print("Target shape:", y.shape)

categorical_cols = X.select_dtypes(

    include=["str", "object"]

).columns
```

```
X_encoded = X.copy()

label_encoders = {}

for col in categorical_cols:

    le = LabelEncoder()

    X_encoded[col] = le.fit_transform(

        X_encoded[col].astype(str)

    )

    label_encoders[col] = le

print("Categorical variables encoded successfully!")

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X_encoded)

print("Feature scaling completed!")

X_train, X_test, y_train, y_test = train_test_split(

    X_scaled,

    y,

    test_size=0.30,

    random_state=42,

    stratify=y

)

print("Training data:", X_train.shape)

print("Testing data:", X_test.shape)

svm_model = SVC(kernel="linear")
```

```
svm_model.fit(X_train, y_train)

print("SVM model trained successfully!")

y_pred_svm = svm_model.predict(X_test)

svm_accuracy = accuracy_score(y_test, y_pred_svm)

svm_cm = confusion_matrix(

    y_test,

    y_pred_svm

)

print("\n==== SVM RESULTS =====")

print("SVM Accuracy:", svm_accuracy)

print("\nSVM Confusion Matrix:")

print(svm_cm)

rf_model = RandomForestClassifier(

    n_estimators=100,

    random_state=42

)

rf_model.fit(X_train, y_train)

print("\nRandom Forest model trained successfully!")

y_pred_rf = rf_model.predict(X_test)

rf_accuracy = accuracy_score(

    y_test,

    y_pred_rf

)
```

```
rf_cm = confusion_matrix(
    y_test,
    y_pred_rf
)

print("\n===== RANDOM FOREST RESULTS =====")

print("Random Forest Accuracy:", rf_accuracy)

print("\nRandom Forest Confusion Matrix:")

print(rf_cm)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

sns.heatmap(
    svm_cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("SVM Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.subplot(1, 2, 2)

sns.heatmap(
    rf_cm,
    annot=True,
```

```
    fmt="d",  
    cmap="Greens"  
)  
plt.title("Random Forest Confusion Matrix")  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.tight_layout()  
plt.show()  
print("\n=====")  
print("MODEL PERFORMANCE COMPARISON")  
print("=====")  
print("SVM Accuracy      :", round(svm_accuracy, 4))  
print("Random Forest Accuracy:", round(rf_accuracy, 4))
```

OUTPUT:

```
Libraries imported successfully!
Dataset loaded successfully!
Dataset shape: (500, 14)

PetType AgeMonths Gender Breed Vaccinated Dewormed Sterilized Health MaturitySize FurLength Fee PhotoAmount VideoAmount AdoptionSpeed
Dog 64 Male Beagle Yes No Yes Healthy Small Medium 732.0 4 0 1
Cat 86 Male Persian No Yes No Healthy Large Medium 1748.0 6 0 2
Dog 33 Female Mixed Breed No Yes Yes Healthy Medium Medium 3924.0 9 2 3
Dog 88 Male Beagle No Yes Yes Healthy Small Medium 423.0 10 0 3
Dog 34 Male Mixed Breed Yes Yes Yes Healthy Large Short 3478.0 10 1 2

Missing values handled successfully!
Features shape: (500, 13)
Target shape: (500,)
Categorical variables encoded successfully!
Feature scaling completed!
Training data: (350, 13)
Testing data: (150, 13)
SVM model trained successfully!

===== SVM RESULTS =====
SVM Accuracy: 0.3333333333333333

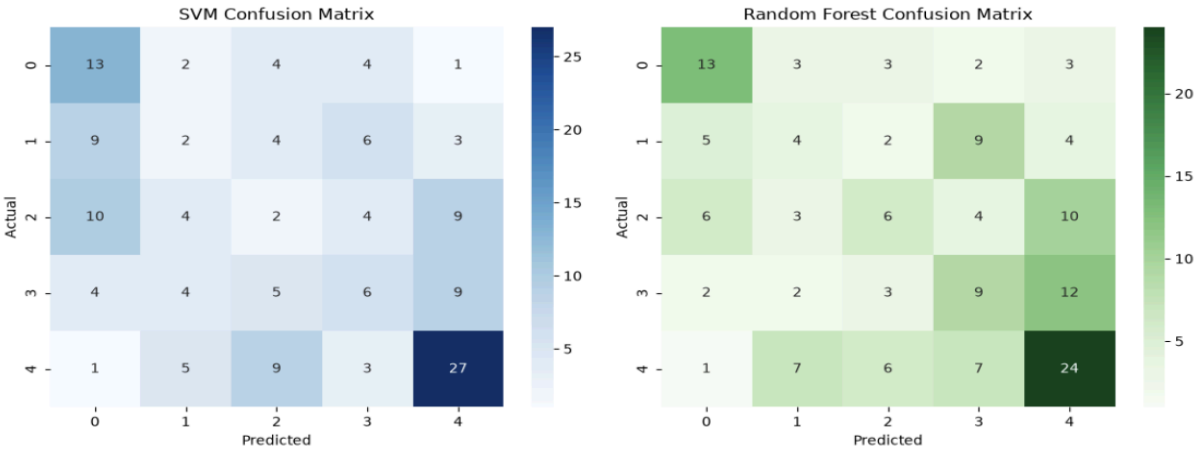
SVM Confusion Matrix:
[[13 2 4 4 1]
 [ 9 2 4 6 3]
 [10 4 2 4 9]
 [ 4 4 5 6 9]
 [ 1 5 9 3 27]]

Random Forest model trained successfully!

===== RANDOM FOREST RESULTS =====
Random Forest Accuracy: 0.3733333333333333

Random Forest Confusion Matrix:
[[13 3 3 2 3]
 [ 5 4 2 9 4]
 [ 6 3 6 4 10]
 [ 2 2 3 9 12]
 [ 1 7 6 7 24]]

=====
MODEL PERFORMANCE COMPARISON
=====
SVM Accuracy : 0.3333
Random Forest Accuracy: 0.3733
```



RESULT:

The SVM and Random Forest classification models were successfully implemented on the Pet Adoption dataset. The models were trained and tested, and their performance was evaluated using accuracy and confusion matrices